

Embedded development and QA teams focused on quality, as software defects in embedded devices can cause life-threatening consequences. By contrast, security was most often a concern with web and commercial applications that handle sensitive customer data.

However, because customers now rely heavily on interconnected web and embedded applications, all development teams should be addressing both code quality and code security.

Code quality

Code quality is a metric that can be used to classify code as good or bad. A code beautifully written, but failing to accomplish the **desired outcome**, can be said to have **low quality**. In short, it must actually meet the customer's needs.

Quality code is defined as code that functions as intended for end-users without any deficiencies. Code is easier to read, understand, and maintain.

Code security

Code security is the framework that provides the security features to secure code from unauthorized users and unauthorized uses.

There are **two types** of code security:

Role based security: This authorizes user.

Code access security: This protects system resources from unauthorized calls.

Importance of Code Quality

1) Making software robust

End-users feel very uncomfortable if they receive a strange message when they do something wrong in software. Here is when robustness comes to play. The software can cope with errors during execution in spite of unusual conditions. High-quality software has clear and understandable error messages for end users.

2) Increasing readability and ease of editing

Good quality of code makes sure that codes are written in such a way that makes them **highly readable**. The use of comments, proper indentation, clear notations, and simplicity in the flow are some factors. Editing code is also a more comfortable job with high-quality code as they are easy to read and implement changes.

3) Making program sustainable

Software is said to be sustainable when it can survive over time with minimal changes. An average software life is accounted to be around six years, but poor software quality does not last even half of this time duration. It's because, with the constant development of operating programs and hardware, change in software increases. It's challenging and sometimes impossible to perform transformations on the poor quality of code.

4) Promotes easy transferability

A high-quality code has high readability and is easy to edit. Another added importance is that code quality practices make the translation of software across platforms straightforward. With numerous platforms for software to work on, it is essential to have direct transferability with minimal changes required.

5) Decreasing technical debt

Software development is itself a high budgeted job with expectations of software to work as long as it can, with minimum error. But software with poor quality is set to fail early unless a significant number of changes are brought into the program repeatedly and hence increasing the technical debt. The extra development work is time and capital consuming, which a high-quality code avoids.

Tools to check code quality

SonarQube

SonarQube is an excellent tool for code quality checks. One of the benefits of SonarQube is that it is an open-source tool. It supports many programming languages, such as C#, Java, C/C++, JavaScript, TypeScript, etc. It supports many integrated development environments (IDEs), including Visual Studio and Eclipse and builds systems like Maven, Gulp, and Gradle etc.

Resharper

Resharper is a Visual Studio add-in that provides additional code editor features and functionality and enhanced support for multiple languages such as C#, JavaScript, TypeScript, CSS, HTML, XML, and XAML. The tool helps you find errors in your code so that they do not make it into production. It also provides code refactoring tools to simply modify existing code while minimizing the potential for bugs to crop up.

FxCop

FxCop is a static code analysis tool. It is a free tool that developers can use to find problems in their code. You can run FxCop by using the command line or the integrated development environment (IDE) of Visual Studio.

FxCop reports coding convention violations, such as naming conventions or capitalization mistakes.

CodeRush

CodeRush helps with code analysis and refactoring. CodeRush enables you to find and fix errors in your code quickly. The powerful code analysis engine automatically detects errors as you type, flags them with squiggles, and offers suggested fixes. CodeRush lets you perform safe automated refactoring like Extract Method, Rename Identifier, Introduce Variable and more.

NDepend

NDepend is a static analyzer that allows programmers to visualize your code and analyze it in-depth. It's a paid tool, but it can be quite helpful for finding out exactly where your code could use some work. NDepend provides a dependency graph to see where each class depends on the others. This will allow you to quickly visualize which classes are dependent on too many other classes—or which ones are not dependent on anything at all!

NDepend will also let you know how much coverage is missing from the project. This helps you find code areas that could use more testing (or any testing at all). Code complexity refers to how difficult it is for someone else to understand your program as they read through it. Using NDepend's visualization tools, you can easily see which parts of your code are simple and easy to understand—and which ones could use some work.

Container

Containers are executable units of software in which application code is packaged, along with its libraries and dependencies, in common ways so that it can be run anywhere, whether it be on desktop, traditional IT, or the cloud.

Containers are small, fast, and portable because unlike a virtual machine, containers do not need include a guest OS in every instance and can, instead, simply leverage the features and resources of the host OS.

A container is basically used to store the entire runtime environment of an application, including all its dependencies like the different libraries and configuration files needed to successfully run it. All of these are bundled together into a single package. This ensures that any process inside the container cannot see any other process or resource outside the container.

For long, the hypervisor based virtualization technology was used to emulate hardware, due to which we could run any OS on top of any other, Linux on Windows, or the other way around. With this technology, both the guest OS and the host OS run with their own kernel. The communication between the guest system and the actual hardware is done with the help of an abstract layer of the hypervisor. This approach actually provides a high level

of isolation as well as security, since all communication between the guest and the host is through the hypervisor. But, this approach is much slower and incurs performance overhead due to hardware emulation. Hence, to reduce this overhead, container virtualization or containerization was introduced, which allows users to run multiple, isolated user space instances using the same kernel.

Containerization helps us run any software reliably when it is moved from one computing environment to another. It is really helpful when moving an application that has been developed, from a developer's desktop to a certification or test environment, and further into production. There are cases when we encounter some unexpected exceptions whenever an application or software is migrated from one environment to another, even when that application works fine in the earlier one. Such issues are also observed when software is moved from a physical machine in a data centre to another virtual machine present in a public or private cloud. If we have the application encapsulated in software containers, then we can get rid of all such issues.

Software containers are really helpful when the supporting environment or platform on which the software has been developed is not identical to the one in which it's implemented. Nowadays, containers are also being used to manage different issues caused due to differences in network topology or different security and storage policies. Hence, containers are designed to solve different application management issues. Overall, containers help us take the DevOps culture a step ahead by increasing the portability of an application. They help to increase the storage and CPU efficiency as well.

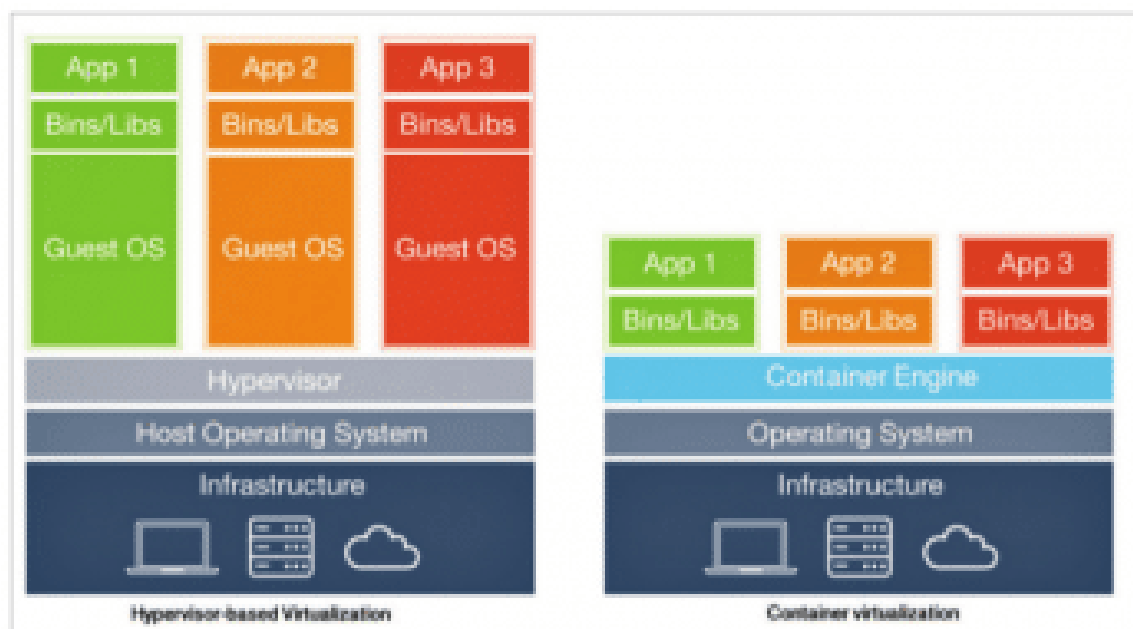


Figure 1: Containers vs. virtual machine architecture

Benefits of containers

- **Lightweight:** Containers share the machine OS kernel, eliminating the need for a full OS instance per application and making container files small and easy on resources.
- **Portable and platform independent:** Containers carry all their dependencies with them, meaning that software can be written once and then run without needing to be re-configured across laptops, cloud, and on-premises computing environments.
- **Supports modern development and architecture:** Due to a combination of their deployment portability/consistency across platforms and their small size, containers are an ideal fit for modern development and application patterns—such as DevOps, serverless, and microservices—that are built are regular code deployments in small increments.
- **Improves utilization:** Containers enable developers and operators to improve CPU and memory utilization of physical machines.

Containerization

Software needs to be designed and packaged differently in order to take advantage of containers—a process commonly referred to as **containerization**.

OR

Containerization is a type of virtualization in which all the components of an application are bundled into a single container image and can be run in isolated user space on the same shared operating system.

When containerizing an application, the process includes packaging an application with its relevant environment variables, configuration files, libraries, and software dependencies. The result is a **container image** that can then be run on a container platform.

Advantages of containerization

1. Containers used in containerization are much more lightweight as they use much fewer resources than virtual machines.
2. Containerization helps to increase the efficiency of CPUs by saving the storage and memory of the system.
3. Containerization leads to high portability.
4. With the help of containerization, different containers can be instantiated precisely when they are required, and they can disappear when no longer required. This helps in freeing up the resources on their hosts when they are not in use.
5. A single server can be used to host a far greater number of containers than virtual machines and, hence, supports sharing.

6. We can track successive versions of a container, inspect the differences, or even roll back to the previous versions using containerization.
7. Containerization leads to greater modularity.
8. It helps us make different changes to the various modules of the application without us having to work on the entire application.

Working principle

How Does Containerization Work?

Containerization works by virtualizing all the required pieces of a specific application into a single unit.

Under the hood, that means containers include all the binaries, libraries, and configuration an app requires. However, containers do NOT include virtualized hardware or kernel resources.

Instead, containers run “on top” of a container runtime platform that abstracts the resources. Because containers just include the basic components and dependencies of an app without additional bloat, they are faster and more lightweight than alternatives like virtual machines or bare metal servers. They also make it possible to abstract away the problems related to running the same app in different environments. If you can provide the underlying container engine, you can run the containerized application.